Tackling the Poor Assumptions of Naive Bayes Text Classifiers

Paper Study Report — Summary, Syllabus Mapping and Project Ideas

Paper	J. D. M. Rennie, L. Shih, J. Teevan and D. R. Karger, "Tackling the Poor Assumptions of Naive Bayes Text Classifiers," *Proc. 20th International Conference on Machine Learning (ICML 2003)*, Washington DC, pp. 616–623.
Course	AI & Machine Learning (M4010100 / M4020100) — Instructor: Dr. Asharaf S
Team	[Name 1] ([Roll No.]) | [Name 2] ([Roll No.])

1. Summary of the Research Paper

Problem. Multinomial Naive Bayes (MNB) is a popular baseline for text classification because it is fast and simple, but it usually loses to stronger classifiers such as the Support Vector Machine (SVM). The authors ask *why* it performs poorly and propose simple fixes that keep its speed. Their key observation is that the MNB decision rule, $l(d) = \text{argmax}_c [\, b_c + \sum_i f_i w_{ci} \,]$, is a **linear classifier** of the same form as the perceptron, logistic regression and a linear SVM. Here f_i is the count of word i and $w_{ci} = \log \theta_{ci}$ is a smoothed maximum-likelihood estimate. MNB's weakness therefore lies in *how it chooses the weights*, not in the form of the boundary.

Fixing systemic errors

(a) Skewed data bias → Complement Naive Bayes (CNB). When classes have unequal amounts of training data, the weights of small classes are biased downward (because log is concave, the expected log of an estimate is smaller than the log of its expected value, $E[\log \theta] < \log E[\theta]$), so the classifier unfairly favours large classes. A small coin-toss example in the paper shows a class with a *lower* true rate of heads winning more often simply because it has more samples. CNB instead estimates each class's parameters from all documents *not* in that class, and assigns a document to the class it matches *least badly*. Each estimate now uses a more even, larger amount of data, which gives more stable weights.

(b) Weight magnitude errors → Weight-normalized CNB (WCNB). The independence assumption double-counts correlated words. For example, "San Francisco" contributes the weight of two words while "Boston" contributes one, so classes with strongly dependent words get larger weight vectors and dominate. The fix is to normalize each class's weight vector by the sum of the absolute values of its weights.

Modelling text better

Real word counts are "bursty": once a word appears in a document, it is likely to appear again. Their distribution has a much heavier tail than a multinomial predicts, close to a power law. The authors borrow three transforms from information retrieval: **(1) log term frequency**, $f' = \log(1 + f)$, which makes power-law counts look more multinomial; **(2) inverse document frequency (IDF)**, which down-weights words that appear in almost every document; and **(3) length normalization**, which scales each document vector to unit length so long documents do not dominate training. Combining these with CNB and weight normalization gives **TWCNB** (Transformed Weight-normalized Complement Naive Bayes).

The complete TWCNB procedure

1. Log term frequency: $d_{ij} \leftarrow \log(d_{ij} + 1)$, where d_{ij} is the count of word i in document j.
2. IDF: $d_{ij} \leftarrow d_{ij} \cdot \log(\text{total documents} / \text{documents containing word } i)$.
3. Length normalization: $d_{ij} \leftarrow d_{ij} / \sqrt{(\sum_k d_{kj}^2)}$.
4. Complement estimate: $\theta_{ci} = (\sum_{j:\, y_j \neq c} d_{ij} + \alpha_i) / (\sum_{j:\, y_j \neq c} \sum_k d_{kj} + \alpha)$, using only documents **outside** class c.
5–6. Weights: $w_{ci} = \log \theta_{ci}$, then normalize: $w_{ci} \leftarrow w_{ci} / \sum_i |w_{ci}|$.
7–8. Classify a test document t as $\text{argmin}_c \sum_i t_i w_{ci}$ (the class whose complement it matches least).

Algorithm adapted from Table 4 of the paper. Steps 1–3 are the text transforms; steps 4 and 6 are the systemic fixes.

Results

Dataset (metric)	MNB	WCNB	TWCNB	SVM
Industry Sector (accuracy)	0.582	0.889	0.923	0.934
20 Newsgroups (accuracy)	0.848	0.857	0.861	0.862
Reuters-21578, micro (P/R breakeven)	0.739	0.782	0.844	0.887
Reuters-21578, macro (P/R breakeven)	0.270	0.548	0.647	0.694

Table 1. Results reported in the paper (its Tables 2 and 3). The SVM used the same text transforms.

TWCNB greatly improves on MNB and comes close to the SVM while remaining much faster and simpler. The largest gains are on the imbalanced datasets (Industry Sector, Reuters), and the jump in Reuters *macro* score shows that small classes benefit most. Among the transforms, length normalization helped most, then the log transform; IDF mattered least. **Limitation:** after these changes the model is no longer a true generative probability model, so techniques such as EM for unlabeled data can no longer be applied.

2. Mapping to the Course Syllabus

The paper touches every module of the syllabus. Table 2 connects each idea in the paper to what we have studied in class.

Idea in the paper	Syllabus topic (Module)	Connection
Multinomial Naive Bayes and the minimum-error (maximum posterior) rule	Supervised learning: Bayesian classification (M2)	The base algorithm being improved. It is also the Module 2 lab exercise: “Implementation of Naive Bayes Classifier.”
Parameter estimates $\theta_{ci} = (N_{ci} + \alpha_i) / (N_c + \alpha)$	Density estimation: Maximum Likelihood (M3)	Word probabilities are smoothed ML estimates (Laplace smoothing via a Dirichlet prior). CNB changes *which data* the estimate uses.
NB viewed as a linear decision boundary	Perceptron (M1); linear and logistic regression (M2); SVM (M4)	Shows that NB, the perceptron, logistic regression and a linear SVM share the same boundary form; they differ only in how the weights are learned.
Skewed data bias; stable weights with more data	Generalization performance; bias–variance trade-off (M2)	A concrete case of estimator bias (via Jensen's inequality) that hurts generalization. CNB trades a slightly different estimate for lower bias and variance on small classes.
Log-TF, IDF and length-normalization transforms	Feature engineering: relevance and feature extraction (M2)	Classic feature engineering: re-scaling features so they fit the model's assumptions. IDF is a relevance weighting of features.
Weight normalization to counter the independence assumption	Feature relevance (M2); capacity control, SRM (M4)	*Our interpretation:* normalization limits how much any class's weight vector can dominate, similar in spirit to controlling model complexity.
Comparison with a linear-kernel SVM	Kernel machines: Support Vector Machines (M4)	The SVM is the “state of the art” benchmark. The paper uses a linear kernel because it performs as well as non-linear kernels on text.
Text classification as a real application; benchmark evaluation	Course outcomes CO3–CO5	Problem analysis (why NB fails), solution design (the fixes) and solution illustration (experiments on three benchmarks).

Table 2. Mapping of the paper's ideas to syllabus modules (M1–M4) and course outcomes.

3. Ideas for Our Project (Brainstorm)

Core reproduction. Implement MNB and TWCNB from scratch in Python (NumPy), following the eight-step procedure in the paper's Table 4. Evaluate them on 20 Newsgroups, which loads directly with scikit-learn, using the paper's setup: 10 random 80/20 splits and mean accuracy. Validate our implementation against scikit-learn's MultinomialNB and ComplementNB, and compare against a linear SVM.

1. **Step-by-step ablation study.** The paper reports only MNB, WCNB and TWCNB, and states in words which transform helps most. We can add one fix at a time (MNB → CNB → WCNB → +log-TF → +IDF → +length norm) and measure each step's contribution, checking the claim that length normalization matters most.
2. **Controlled imbalance experiment.** 20 Newsgroups is balanced, so it hides CNB's main benefit. We can artificially subsample some classes (for example, imbalance ratios of 1:1, 1:5 and 1:20) and plot how the MNB–CNB gap grows with imbalance. We can also simulate the paper's coin-toss example to show the bias numerically.
3. **Weight-stability plot.** Recreate the paper's Figure 1: track the weights of a few discriminative words as training data grows from 10 to 1000 documents per class, for both MNB and CNB.
4. **Stronger baselines and speed.** Compare TWCNB with logistic regression and a linear SVM on accuracy *and* training time. The paper's main selling point is “nearly as good, much faster,” so timing is worth measuring.
5. **Hyperparameter study.** The paper fixes the smoothing value $\alpha = 1$ but notes that tuning it helps. Cross-validate α, and also try the constant d in the $\log(d + f)$ transform.
6. **New datasets.** Test whether the fixes still hold on different text: short texts (SMS spam or tweets), a highly imbalanced dataset, or an Indian-language news classification dataset (for example, Malayalam or Hindi).
7. **CNB versus other imbalance remedies.** Compare CNB with class weighting and oversampling of small classes, to see whether the complement trick is better than generic fixes.
8. **Discussion points.** What is lost when NB stops being a probabilistic model (calibrated probabilities, EM for semi-supervised learning)? Are the fixes still relevant now that transformer models exist, for example as a fast, interpretable baseline?

Suggested plan. Items 1–3 form the minimum project and directly test the paper's claims. Items 4–6 add our own contribution for the “observations” part of the presentation. The code will be published in a public GitHub repository, with a short blog post describing the experience, in line with the course project guidelines.

Planned experimental setup

Component	Plan
Datasets	20 Newsgroups (balanced, as in the paper); an artificially imbalanced version of it; one additional dataset from idea 6.
Methods	MNB, CNB, WCNB and TWCNB (our NumPy implementation); scikit-learn MultinomialNB and ComplementNB as reference checks.
Baselines	Logistic regression and linear SVM, trained on the same TF-IDF features.
Metrics	Accuracy and macro-F1 (macro-F1 shows small-class performance); training and prediction time.
Protocol	10 random stratified 80/20 splits; report mean ± standard deviation; α tuned by cross-validation on training data only.
Tools	Python, NumPy, scikit-learn, Matplotlib; code in a public GitHub repository.

References

[1] J. D. M. Rennie, L. Shih, J. Teevan and D. R. Karger, “Tackling the Poor Assumptions of Naive Bayes Text Classifiers,” *Proc. ICML*, 2003, pp. 616–623.

[2] A. McCallum and K. Nigam, “A Comparison of Event Models for Naive Bayes Text Classification,” *AAAI-98 Workshop on Learning for Text Categorization*, 1998.

[3] P. Domingos and M. Pazzani, “Beyond Independence: Conditions for the Optimality of the Simple Bayesian Classifier,” *Proc. ICML*, 1996.

[4] T. Joachims, “Text Categorization with Support Vector Machines: Learning with Many Relevant Features,” *Proc. ECML*, 1998.